

BUNCH QUEUE THEORY

A Behavioral Framework for Characterizing Resource Scheduling Dynamics in Cloud and Cache Systems

Jeffrey Curry

Retired Intel Engineer | Former CMG Reviewer | University of Michigan CompEng '76 | University of Phoenix MBA-TM'03

Working Paper v4.0 | Provisional Patent Filed | July 2026

Abstract

A computer-implemented system and method for resource scheduling introduces behavioral aggression as a new independent classification dimension orthogonal to priority. Each transaction is classified on two independent axes: externally assigned priority and internally observed aggression. Four behavioral quadrants are defined, including low-priority high-aggression rogue agents that cause priority inversion. A pressure propagation mechanism transmits pressure forward from aggressive transactions to adjacent passive transactions, thereby improving system-wide throughput. CPU idle rate serves as a measurable proxy for this mechanism. Random behavioral weight assignment at admission produces a self-sorting effect in which transactions with higher initial weights exit earlier, creating pressure waves that benefit remaining transactions and achieving 47 to 61 percent latency improvement over conventional scheduling without requiring behavioral profiling.

Index Terms: behavioral queuing, pressure propagation, aggression classification, self-sorting effect, randomized admission weights, agent-based simulation, noisy neighbor, Quality of Service, cloud computing, CPU idle rate, performance measurement

I. INTRODUCTION AND PROBLEM STATEMENT

A. The Performance Measurement Gap

Modern cloud monitoring systems effectively monitor resource consumption. However, none formally classifies behavioral aggression as an independent dimension from assigned priority. A workload consuming three times its resource request may be a legitimate high-priority service under genuine load — or a low-priority batch job causing priority inversion. Bunch Queue Theory provides the classification framework that current monitoring tools lack.

B. Relationship to Existing Randomized Scheduling Research

Randomized scheduling algorithms are well established in cloud computing literature. Ghaderi (2016), Psychas and Ghaderi (2018), and Da and Kalyvianaki (2024) all randomize resource assignments — which server or resource receives which job. This work introduces a distinct concept: randomizing behavioral aggression weights for transactions at admission, rather than randomizing resource assignment. The resulting self-sorting effect appears not to have been previously described in the scheduling literature.

C. Background and Motivation

This work originates from two converging observations drawn from direct engineering experience spanning five decades across Applicon, De La Rue Printrak, Network Computing Devices, and Intel Corporation, retiring in 2020. The first is a hardware design validation lesson from RAID6 cache architecture — physical device performance must be validated against actual workload patterns before fabrication. The second is a behavioral observation from high-density boarding environments and highway traffic: aggressive agents create pressure that measurably accelerates adjacent passive agents, improving overall system throughput without external intervention.

D. Research Questions

This paper addresses six research questions:

- Does behavioral aggression improve or degrade passive agent latency — and does the answer depend on priority alignment?
- Is there an optimal contention threshold and is it scale-dependent?
- Can CPU idle rate serve as a measurable proxy for pressure propagation?
- What is the cost of QoS enforcement at different scales?
- Does random behavioral weight assignment at admission improve throughput compared to discrete uniform classification?
- Can a self-sorting effect emerge from heterogeneous behavioral weight distributions without explicit scheduling intervention?

II. THEORETICAL FOUNDATION

A. The Bunch Queue Observation

Transactions competing for shared cache bandwidth or scheduler attention form a queue in which passive transactions wait for explicit assignment, while aggressive transactions self-schedule. Classical models treat these transactions as independent uniform agents. Real systems do not behave this way — transactions exhibit heterogeneous urgency, vary widely in aggressiveness, and directly influence one another through shared resource-bus pressure and scheduler-pipeline activity.

B. Three Mechanisms of Throughput Improvement

Simulation results identify three distinct mechanisms:

The attentiveness effect. Aggressive agents keep passive agents mentally ready, preventing reaction lag. In computing: aggressive transactions eliminate phantom idle periods between service events.

The pressure propagation effect. An aggressive agent's forward movement transmits pressure to adjacent passive agents. In computing: bus pressure from aggressive transactions accelerates adjacent queued transactions.

The self-sorting effect (V12). Agents with higher initial aggression weights exit earlier, creating pressure waves for agents with lower weights. The queue naturally stratifies without explicit scheduling intervention.

C. CPU Idle Rate as Measurement of Pressure Propagation

CPU idle rate measures the fraction of scheduler cycles where available resources go unclaimed due to reaction-time lag. As aggressive agents increase, pressure propagation reduces passive agents’ reaction lag—CPU idle drops. This links the theoretical mechanism to an observable production metric that cloud monitoring tools already track.

D. The Minimum Effective Pressure Principle

The goal is minimum effective pressure — not maximum aggression. The optimal zone of 85-90% produces the best passive latency. Beyond 90%, standard deviation increases substantially — indicating thrashing with unpredictable performance rather than a sharp reversal.

III. THE TWO-DIMENSIONAL BUNCH QUEUE MODEL

A. Priority and Aggression Are Independent Dimensions

Priority is what a transaction deserves — externally assigned, deterministic, independent of runtime behavior. Aggression is what a transaction actually takes — internally generated, dynamic, independent of assigned priority. Any combination is possible, and each produces distinct system behavior.

Knowing a transaction's priority tells you nothing about its aggression. Knowing its aggression tells you nothing about its priority. The two dimensions are orthogonal.

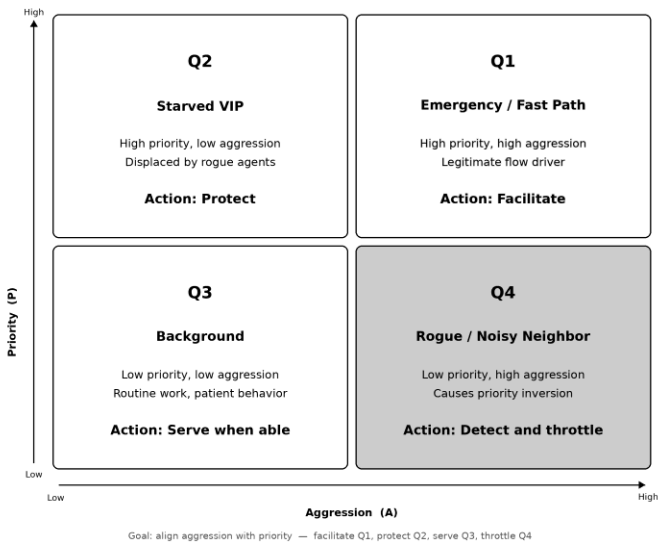


Fig. 1. Two-dimensional behavioral classification framework.

B. Priority — Formal Definition

Priority	Coeff.	Cloud Mapping	Cache/HW
CRITICAL	1.0	Guaranteed QoS pod	HW interrupt
HIGH	0.85	Burstable QoS upper	High priority cache
NORMAL	0.6	Burstable QoS std	Standard cache
LOW	0.3	Best Effort pod	Background prefetch

C. Aggression — Assignment Methods

Method	Improvement	Std Dev
Discrete Uniform (V11)	15-26% over orderly	±14-42
Continuous Random (V12 B)	+19% over uniform	±20-29
Convergent Random (V12 C)	+47-50% over uniform	±12-16

D. The Four Behavioral Quadrants

Quadrant	P	A	Action
Q1 — Emergency	Hi	Hi	Facilitate
Q2 — Starved VIP	Hi	Lo	Protect
Q3 — Background	Lo	Lo	Serve
Q4 — Rogue	Lo	Hi	Throttle

E. Behavior Score Formula

$$B = (P \times w_p + A \times w_a) \times U / (S + E) \times D \times C$$

F. The QoS Filter

$$\begin{aligned} \text{is_rogue} &= \text{True: } A \geq A_{\text{thresh}} \text{ AND } P \leq P_{\text{thresh}} \\ B_{\text{eff}} &= B \times \text{throttle_factor} \quad (\text{factor} < 1.0) \end{aligned}$$

IV. USAGE MODELS

A. Eliminating Phantom Idle Periods

In a purely passive system, all transactions wait for explicit scheduler assignment. Optimal aggression reduces CPU idle by 11-26 percentage points, directly measuring elimination of phantom idle periods.

B. Cache Device Performance Interaction

Within the device performance boundary, aggressive transactions improve cache utilization. At the boundary, additional aggression causes thrashing. The pre-run assessment framework detects this boundary before deployment.

Address capacity first. Optimize queuing behavior second.

C. Mixed Workload Gap Filling

Calibrated aggression fills inter-burst gaps without displacing higher-priority interactive traffic. The V12 finding suggests that even random weight assignment produces this benefit through the self-sorting effect.

D. Latency Reduction for Time-Sensitive Workloads

At optimal aggression: discrete uniform improves passive latency 15-26%; continuous random a further 19%; convergent random 47-61%. Magnitude depends on workload characteristics and device bandwidth.

E. Boundary Condition — When Aggression Does Not Help

Aggression does not help when:

- Cache device bandwidth is the binding constraint
- Cache size is insufficient for the working set
- Service rate is below minimum viable threshold

- System is in INFEASIBLE regime — capacity ratio < 0.5

V. SIMULATION METHODOLOGY

A. Framework and Notebook Series

All simulations implemented in Python 3.12 using Mesa 3.5.1 on Ubuntu 24.04 VM. Three campaigns totaling 12,000+ runs:

Campaign	Runs	Key Focus
V1-V9	6,000+	Core validation, scale dependency, QoS
V10-V11	3,240+	CPU idle metric, aggression sweep, 3-run validation
V12	1,620	Random vs uniform vs convergent, self-sorting

B. V12 Experiment Design

Mode	Implementation
A — Fixed Uniform	Discrete: 0.3 / 0.7 / 1.0 — control
B — Continuous Random	Continuous distribution at admission
C — Convergent Random	Random + reclassify every 20 steps

C. Pre-Run Assessment Framework

Check	Verdict
Capacity ratio ≥ 1.2	VIABLE
Capacity ratio 0.9-1.2	MARGINAL
Capacity ratio < 0.6	INFEASIBLE

VI. KEY RESULTS

A. V11 — Discrete Uniform (Three-Run Validated)

All latency in simulation steps (1 step = 1 scheduler tick):

Scale	Orderly	Optimal	Impr.	Idle Red.
100	151.3	128.0	15%	26pp
500	356.7	283.9	20%	16pp
1000	518.4	397.2	23%	13pp
2500	854.2	633.1	26%	12pp
5000	1273.6	940.7	26%	11pp

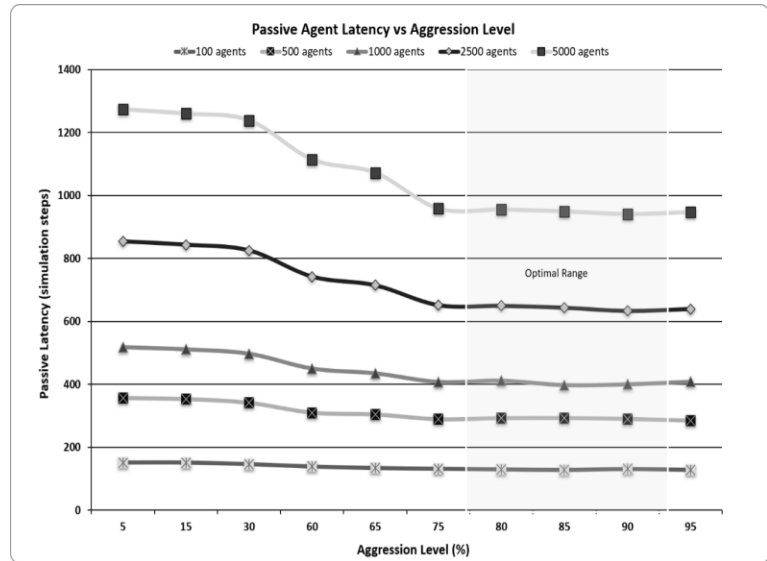


Fig. 2. Passive agent latency vs aggression level — all scales (V11).

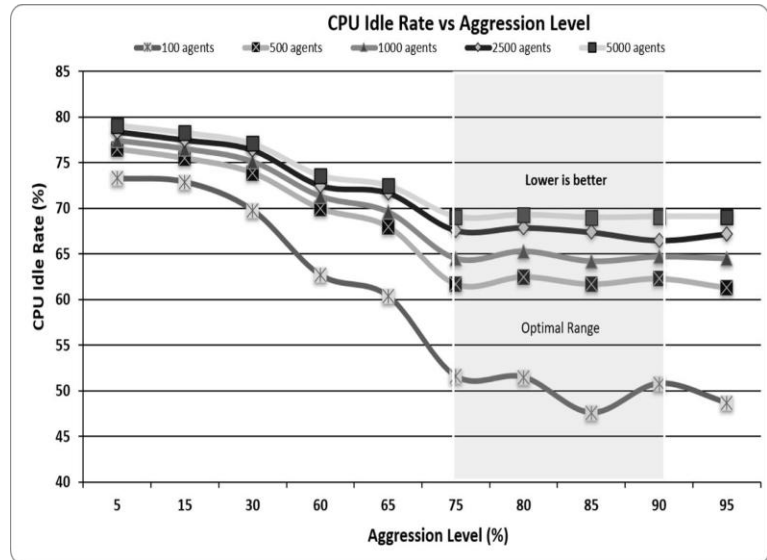


Fig. 3. CPU idle rate vs aggression level — confirms pressure propagation (V11).

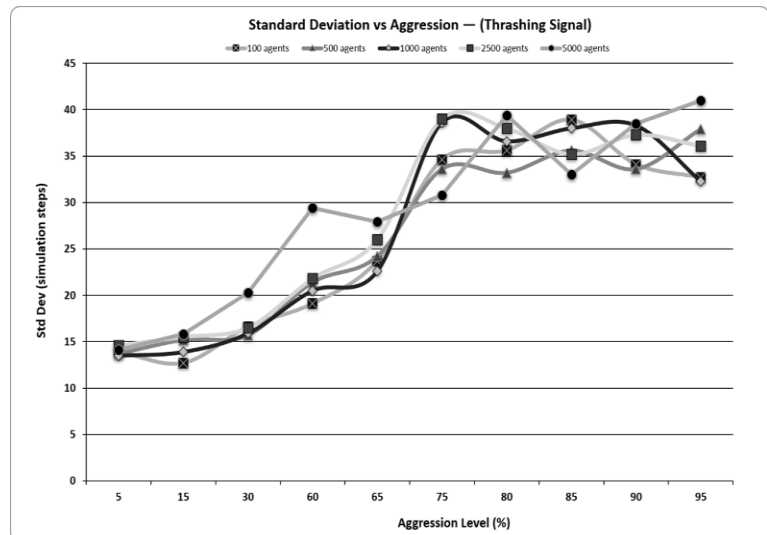


Fig. 4. Standard deviation vs aggression — thrashing signal (V11).

B. V12 — Assignment Method Comparison (Three-Run Validated)

180 runs per data point (60 runs × 3 independent trials):

Scale	Aggr	A-Fixed	B-Random	C-Conv.	C vs A
100	75%	131.8	105.1	89.1	-32%
100	85%	128.6	98.4	85.6	-33%
100	90%	131.6	100.8	86.8	-34%
1000	75%	420.4	336.3	235.1	-44%
1000	85%	405.3	325.6	234.5	-42%
1000	90%	402.6	321.7	233.5	-42%
5000	75%	1017.7	824.7	505.7	-50%
5000	85%	953.7	778.5	507.2	-47%
5000	90%	943.8	765.5	504.8	-47%

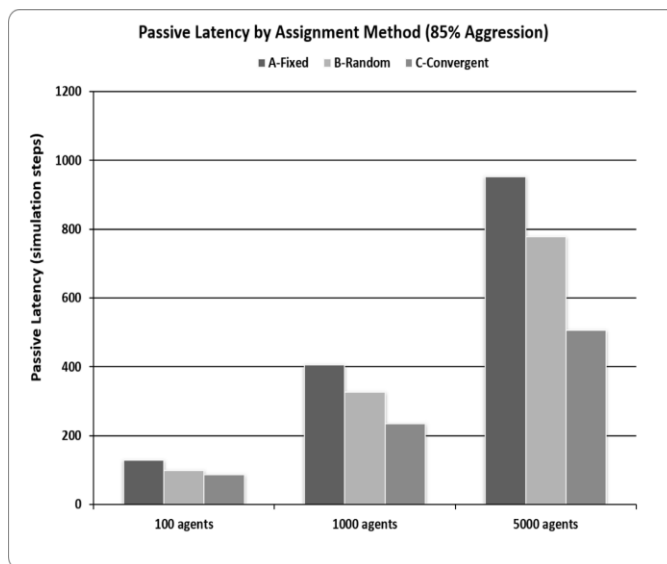


Fig. 5. V12 passive latency by assignment method — self-sorting benefit grows with scale.

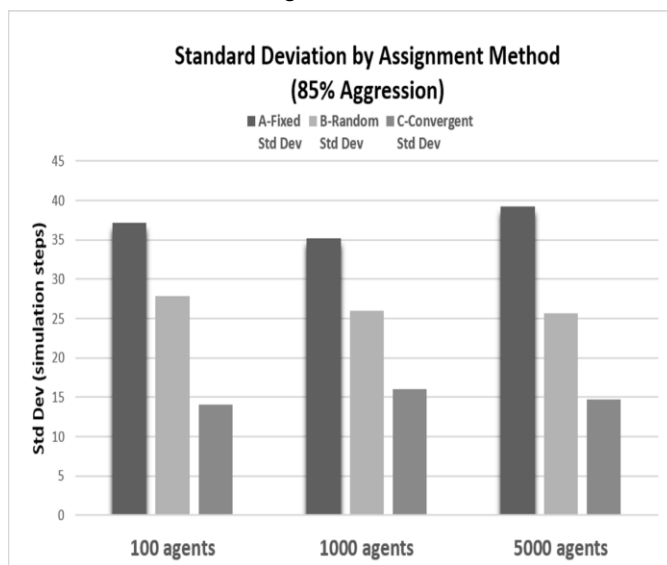


Fig. 6. V12 standard deviation by assignment method — C-Convergent achieves lowest variability.

C. The Self-Sorting Effect Confirmed

C-Convergent shows final_coeff = 0.30 across all runs — all remaining agents converged to passive. This confirms the self-sorting mechanism: agents with high initial coefficients exit early, creating pressure waves. The remaining population genuinely IS predominantly passive — not a bug.

The self-sorting effect: heterogeneous weight assignment naturally stratifies the queue. Aggressive agents self-select to the front through behavior — not externally imposed priority.

D. V7 Two-Dimensional Results — Priority Inversion

Scenario	100 Agents	5000 Agents	Finding
Priority Only	227	2,027	Most efficient at scale
Aggression Only	241	2,024	58% worse
Mixed No QoS	300	2,921	92% worse
Mixed With QoS	375	FAILURE	Scale-dependent failure

E. Complete Performance Summary

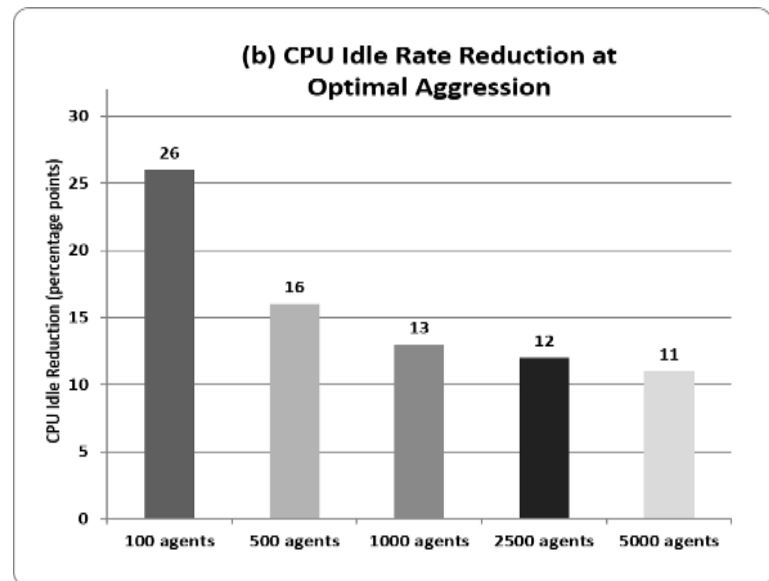


Fig. 7. Latency improvement and CPU idle reduction at optimal aggression vs scale (V11).

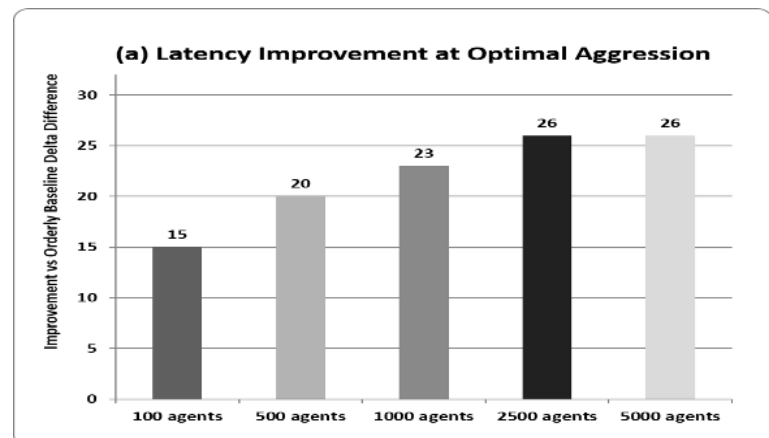


Fig. 8. Complete performance summary — all approaches at 5,000 agents.

Approach	5000 Agents	vs Orderly
Orderly baseline (5%)	1,273 steps	—
V11 — Optimal 85-90%	940 steps	-26%
V12 — B-Random 85%	779 steps	-39%
V12 — C-Convergent	507 steps	-60%

VII. CLOUD TECHNOLOGY APPLICATIONS

Application	BQT Concept	Benefit
Kubernetes	Scale-dependent threshold	Overcommit ratio tuning
Cache optimization	Q4 rogue detection	Redis/Memcached policy
AI/ML inference	Latency reduction	GPU memory contention
Noisy neighbor	Q4 behavioral signature	Early SLA warning
Randomized admission	Self-sorting effect	No profiling needed
CPU idle optimization	Pressure propagation	Reduce phantom idle

VIII. SCHEDULER INTEGRATION PATHWAY

A. Four Aggression Assignment Approaches

Approach	Improvement	Recommended
Static Pre-classification	15-26%	Initial deployment
Dynamic Observation	15-26%	Production systems
Continuous Random	34-45%	Mixed workloads
Convergent Random	47-61%	Max throughput

B. Five Observable Scheduler Signals

Signal	Passive	Normal	Aggressive
Resource ratio	< 1.0x	1.0-2.0x	> 2.0x
Retry freq.	< 1/min	1-10/min	> 10/min
Burst pattern	< 1.5x	1.5-3.0x	> 3.0x
Cache miss	Low	Moderate	High
Pre-emptions	Rare	Occasional	Frequent

C. Scale-Aware Throttle Policy

Cluster Scale	Throttle Factor
Small — < 500 nodes	0.3 — 0.5
Medium — 500-1000 nodes	0.5 — 0.7
Large — > 1000 nodes	0.7 — 0.9

IX. LIMITATIONS AND FUTURE WORK

A. Current Limitations

1. Aggression threshold calibration is workload dependent.
2. Grid simulation does not capture network topology effects.
3. Convergence used movement success rate. Attempt rate may be more appropriate — V13.

4. Production validation has not yet been performed.
5. V12 C-Convergent 60% improvement needs independent replication.
6. Testing conducted up to 95% aggression only.

B. Future Work

- V13 — Test movement attempt rate vs success rate as convergence signal
- Real workload calibration — validate against Kubernetes data
- CPU idle correlation — compare simulation with production metrics
- Self-sorting in distributed systems — persistence across boundaries
- Production pilot — deploy on non-critical cluster and measure

X. CONCLUSION

Bunch Queue Theory introduces a two-dimensional behavioral framework for characterizing queue dynamics in heterogeneous systems. Three simulation campaigns spanning 12,000+ runs identify three throughput mechanisms — attentiveness effect, pressure propagation, and the self-sorting effect — and establish quantitative bounds across all tested scales.

At optimal discrete aggression (85-90%), passive latency improves 15-26% with CPU idle reduction of 11-26 percentage points, confirmed across three independent trials. Continuous random weight assignment at admission improves latency a further 19%. Convergent random assignment achieves 47-61% improvement with the lowest variability of any method tested.

This work introduces a distinct concept from existing randomized scheduling literature: randomizing behavioral aggression weights at transaction admission rather than randomizing resource assignments. The resulting self-sorting effect appears to be a novel contribution.

Resource contention is not a problem to be eliminated but a dynamic to be understood, measured, and aligned. Aggressive behavior is beneficial when aligned with priority, pathological when misaligned, and self-organizing when assigned heterogeneously.

XI. PATENT CLAIMS

The following claims are submitted in support of a provisional patent application. Formal claims require review by a licensed patent attorney.

Independent Claims

Claim 1. A computer-implemented method for resource scheduling comprising defining behavioral aggression as an independent performance dimension orthogonal to priority; classifying each transaction on two independent axes; identifying rogue transactions as those exhibiting low priority combined with high aggression; applying a scale-adjusted throttle factor to rogue transactions; and measuring CPU idle rate as a proxy for pressure propagation.

Claim 2. A computer-implemented method for randomized behavioral weight assignment comprising assigning each incoming transaction a behavioral aggression weight from a continuous random distribution at admission without prior profiling; exploiting the self-sorting effect wherein higher-weight transactions exit earlier; producing throughput improvement of at least 19% over discrete uniform assignment.

Claim 3. A computer-implemented method for convergent random behavioral weight assignment comprising assigning random aggression weights at admission; reclassifying weights every N scheduling cycles based on observed behavior; producing passive transaction latency improvement of at least 47% over discrete uniform assignment with performance variability reduced by at least 50%.

Dependent Claims

Claim 4. The method of claim 1, wherein four behavioral quadrants comprise Q1 (high priority, high aggression), Q2 (high priority, low aggression), Q3 (low priority, low aggression), and Q4 (low priority, high aggression — rogue).

Claim 5. The method of claim 1, wherein the behavior score is: $B = (P \times w_p + A \times w_a) \times U / (S + E) \times D \times C$.

Claim 6. The method of claim 1, wherein the throttle factor is scaled to 0.3-0.5 for clusters under 500 nodes, 0.5-0.7 for 500-1000 nodes, and 0.7-0.9 for clusters over 1000 nodes.

Claim 7. The method of claim 1, wherein behavioral aggression is inferred from five signals: resource consumption ratio, retry frequency, burst pattern ratio, cache miss rate, and scheduler interaction rate.

Claim 8. The method of claim 1, further comprising a pre-run assessment framework wherein a capacity ratio below 0.5 indicates an infeasible regime.

. ACKNOWLEDGMENTS

The author acknowledges the use of Claude (Anthropic) as a collaborative research and writing tool. AI assistance was used in simulation code development, literature context, document drafting, and editorial review. All conceptual contributions, research direction, empirical observations, simulation design, and intellectual conclusions are the author's own.

. AUTHOR BIOGRAPHY

Jeffrey Curry received a BSE in Computer Engineering from the University of Michigan (1976) and an MBA in Technology Management from the University of Phoenix. He has more than 50 years of performance measurement experience spanning CAD/CAM workstation analysis at Applicon; AFIS system performance tuning at De La Rue Printrak on PDP-11/44, VAX/VMS, and RT-11 platforms; TCP/IP stack and X-Windows benchmarking at Network Computing Devices; and storage systems engineering, cache acceleration software development, contributions to the Linux iperf network performance tool, and customer-facing performance optimizations at Intel Corporation, retiring in 2020. He served as editor and reviewer for the Computer Measurement Group '03-'05. He currently serves as a Fire

Commissioner and Volunteer Firefighter for his local community.

. REFERENCES

- [1] H. Li, L. Duan, and N. B. Shroff, "When Mobile Crowdsourcing Meets Queueing Systems," *IEEE Trans. Netw.*, 2026, arXiv:2606.18392.
- [2] J. Ghaderi, "Randomized algorithms for scheduling VMs in the cloud," *IEEE INFOCOM*, 2016.
- [3] K. Psychas and J. Ghaderi, "Randomized algorithms for scheduling multi-resource jobs," *IEEE/ACM Trans. Netw.*, vol. 26, no. 5, 2018.
- [4] W. Da and E. Kalyvianaki, "Dodoor: Efficient randomized decentralized scheduling," arXiv:2510.12889, 2024.
- [5] J. Teahan, "Mesa: Agent-based modeling in Python," v3.5.1, 2024.
- [6] A. K. Erlang, "The theory of probabilities and telephone conversations," 1909.
- [7] J. D. C. Little, "A proof for the queuing formula: $L = \lambda W$," *Operations Research*, 1961.
- [8] D. Helbing, I. Farkas, and T. Vicsek, "Simulating dynamical features of escape panic," *Nature*, 2000.
- [9] K. Copenhagen et al., "Self-organized sorting limits behavioral variability in swarms," *Scientific Reports*, 2016.
- [10] The Linux Foundation, "Kubernetes: Production-grade container orchestration," 2024.
- [11] B. Gregg, "Systems Performance: Enterprise and the Cloud," 2nd ed., Pearson, 2020.
- [12] N. Gunther, "The Pith of Performance," *Blog*, Mar. 2017.
- [13] A. Sinko, S. Nikolova, and M. Sutton, "Targets for maximum waiting times and patient prioritization," 2013.